

Chapter-3: Operators and Expressions

1. Define operator and expression. [2018]

Answer: In programming, an operator is a symbol or keyword that represents a specific operation or action to be performed on one or more operands. Operands are the values or variables on which the operator acts.

Example: Arithmetic operator (+, -, *, /, %), sizeof operator, etc.

An expression is a combination or sequence of operators, operands, and other programming elements that reduces to a single value. Expressions can be simple or complex, and they can involve arithmetic, logical, and other operations.

Example: `int x = 10, y = 20;`

`int sum = x + y;` //Here, `x + y` is an expression that will reduce to a single value 30.

2. What are the different types of operators used in C language? [2022, 2019, 2018, 2016]

Answer: In C, operators are symbols or keywords that perform specific operations on one or more operands. They are classified into different categories based on their functionalities. The main types of operators used in the C language are:

- **Arithmetic Operators:** These operators are used for basic arithmetic calculations, such as addition, subtraction, multiplication, division, and modulo division.
Example: +, -, *, /, %
- **Relational Operators:** Relational operators are used for making comparisons between values or variables. They return a true (1) or false (0) result.
Example: >, <, >=, <=, ==, !=
- **Logical Operators:** Logical operators are used for combining or negating logical expressions. They are used in conditional statements and loops.
Example: && (logical AND), || (logical OR), ! (logical NOT)
- **Bitwise Operators:** Bitwise operators perform operations at the bit level of data. They manipulate individual bits in variables.
Example: & (bitwise AND), | (bitwise OR), ^ (bitwise XOR), ~ (bitwise negation/1's complement), << (left shift), >> (right shift)
- **Assignment Operators:** These operators are used to assign values to variables.
Example: =, +=, -=, *=, /=, %=, <<=, >>=, &=, |=, ^=
- **Increment and Decrement Operators:** Increment and decrement operators are used to increase or decrease the value of a variable by 1.
Example: ++, --

- **Conditional Operator (Ternary Operator):** The conditional operator is a shorthand version of an if-else statement. It is used to make decisions based on a condition.
Example: (condition) ? expression1 : expression2;
- **Comma Operator:** The comma operator is used to separate expressions and evaluates all expressions.
Example: expr1, expr2, expr3

These operators are fundamental to performing various computations, comparisons, and logical operations in C programs. Understanding how each type of operator works is essential for writing efficient and effective C code.

3. Describe the logical and relational operators used in C language. [2023, 2020, 2018]

Answer: Logical operators are used to combine or negate logical expressions, and they return a true (1) or false (0) result. These operators are typically used in conditional statements (if, else if, switch) and loops (while, do-while, for) to control the flow of the program based on conditions.

- **Logical AND (&&):** This operator returns true if both the operands are true.
- **Logical OR (||):** This operator returns true if at least one of the operands is true.
- **Logical NOT (!):** This operator reverses the logical value of the operand. If the operand is true, it becomes false, and if the operand is false, it becomes true.

Example: Considering `int x = 5, y = 2, z = 0;` Now, output of the logical operators will be as follows:

Logical Expression	Output
<code>(x>4) && (y>1)</code>	True (1)
<code>(x<4) && (y>1)</code>	False (0)
<code>(x>6) (y>1)</code>	True (1)
<code>(x>6) (y>3)</code>	False (0)
<code>!z</code>	True (1)

Relational operators are used to compare values or variables and return a true (1) or false (0) result based on the comparison. These operators are often used in conditional statements to make decisions based on the relationship between values.

- **Greater than (>):** Returns true if the left operand is greater than the right operand.
- **Less than (<):** Returns true if the left operand is less than the right operand.
- **Greater than or equal to (>=):** Returns true if the left operand is greater than or equal to the right operand.
- **Less than or equal to (<=):** Returns true if the left operand is less than or equal to the right operand.

- **Equal to (==):** Returns true if the left operand is equal to the right operand.
- **Not equal to (!=):** Returns true if the left operand is not equal to the right operand.

Example: Considering `int x = 5, y = 2`; Now, output of the relational operators will be as follows:

Expression	Output
<code>x > y</code>	True (1)
<code>x < y</code>	False (0)
<code>x >= y</code>	True (1)
<code>x <= y</code>	False (0)
<code>x == y</code>	False (0)
<code>x != y</code>	True (1)

4. Explain the logical and bitwise operators with example. [2016]

Answer: Logical operators are used to combine or negate logical expressions, and they return a true (1) or false (0) result. These operators are typically used in conditional statements (if, else if, switch) and loops (while, do-while, for) to control the flow of the program based on conditions.

- **Logical AND (&&):** This operator returns true if both the operands are true.
- **Logical OR (||):** This operator returns true if at least one of the operands is true.
- **Logical NOT (!):** This operator reverses the logical value of the operand. If the operand is true, it becomes false, and if the operand is false, it becomes true.

Example: Considering `int x = 5, y = 2, z = 0`; Now, output of the logical operators will be as follows:

Logical Expression	Output
<code>(x > 4) && (y > 1)</code>	True (1)
<code>(x < 4) && (y > 1)</code>	False (0)
<code>(x > 6) (y > 1)</code>	True (1)
<code>(x > 6) (y > 3)</code>	False (0)
<code>!z</code>	True (1)

Bitwise operators perform operations at the bit level of data. They manipulate individual bits in variables, which are composed of sequences of 0s and 1s. Bitwise operators are used to perform low-level bit manipulation tasks or to access individual bits within data.

- **Bitwise AND (&):** Performs a bitwise AND operation on the binary representation of the operands. Each bit in the result is set to 1 only if both corresponding bits in the operands are 1.
- **Bitwise OR (|):** Performs a bitwise OR operation on the binary representation of the operands. Each bit in the result is set to 1 if at least one corresponding bit in the operands is 1.



- **Bitwise XOR (^):** Performs a bitwise exclusive OR (XOR) operation on the binary representation of the operands. Each bit in the result is set to 1 if exactly one corresponding bit in the operands is 1.
- **Bitwise NOT/1's Complement (~):** Performs a bitwise NOT operation on the binary representation of the operand. It changes all 0s to 1s and all 1s to 0s.
- **Left Shift (<<):** Shifts the bits of the left operand to the left by a specified number of positions.
- **Right Shift (>>):** Shifts the bits of the left operand to the right by a specified number of positions.

Example: Suppose we have two variables $a = 12$ and $b = 10$ with binary representations '1100' and '1010', respectively. We can apply bitwise operators to manipulate the individual bits:

Bitwise Operators	Output in Binary
$a \& b$	1000
$a b$	1110
$a \wedge b$	0110
$\sim a$	0011
$a \ll 2$	110000
$a \gg 1$	0110

In this example, bitwise operators are used to perform bitwise operations on 'a' and 'b', yielding different binary results for each operation. Bitwise operations are commonly used in low-level programming and when dealing with specific hardware interactions where individual bits are significant.

5. What are the uses of sizeof and conditional operator? [2017]

Answer: The sizeof operator in C is used to determine the size (in bytes) of a data type, a variable, or an expression. It is a compile-time operator that provides valuable information about the memory occupied by a particular data type or variable. The sizeof operator is particularly useful in scenarios where we need to allocate memory dynamically, work with arrays, or perform low-level memory manipulations.

Example: `printf("%d", sizeof(int));` //Output: 4

Uses of sizeof operator:

- **Dynamic Memory Allocation:** When using dynamic memory allocation functions like `malloc()` or `calloc()`, we often need to specify the size of memory to be allocated. The sizeof operator helps to calculate the appropriate size based on the data type.
- **Array Size:** In C, arrays don't store their size, so sizeof can be used to find the size of an array to iterate over its elements correctly.

- **Working with Pointers:** The sizeof operator is useful when working with pointers, as it allows us to allocate memory for data correctly or calculate the size of data being pointed to.
- **File I/O:** In file handling, sizeof helps to determine the size of data to be read from or written to a file.

The conditional operator, also known as the ternary operator, is a shorthand way to write simple if-else statements in a single line. It takes two expressions and evaluates a condition. If the condition is true, the first expression is returned; otherwise, the second expression is returned. The conditional operator is commonly used for conditional assignments or in expressions where we need to choose between two values based on a condition.

Example: `int A = 6, B = 3;`

`(A>B)?printf("A max"):printf("B max"); //Output: A max`

Uses of Conditional Operator:

- **Conditional Assignments:** The conditional operator is often used for conditional assignments, where a variable is assigned different values based on a condition.
- **Simplifying If-Else Statements:** Instead of writing multi-line if-else statements, we can use the conditional operator to make the code more concise.
- **Setting Default Values:** The conditional operator can be used to set default values for variables if certain conditions are not met.
- **Choosing Between Two Values:** In expressions, the conditional operator helps to choose between two values based on a condition, without the need for an explicit if-else block.

Both sizeof operator and the conditional operator are valuable tools that help in writing efficient and concise C code. They improve code readability and make it easier to work with data types and conditions in various scenarios.

6. With example describe the rules for ++ and -- operator. [2021]

Answer: The ++ and -- operators in C are known as increment and decrement operators, respectively. They are used to increase or decrease the value of a variable by one. The rules for using these operators are as follows:

Increment Operator (++):

- The ++ operator is used to increase the value of a variable by one.
- It can be used as a prefix (++x) or a postfix (x++) operator.
- When used as a prefix operator, the variable is incremented first, and then the updated value is used.

Example: `int x = 5;`

`int r1 = ++x; //r1=6 and x=6`

- When used as a postfix operator, the current value of the variable is used first, and then the variable is incremented.

Example: `int x = 10;`

`int r2 = x++; //r2=10 and x=11`

Decrement Operator (--):

- The -- operator is used to decrease the value of a variable by one.
- It can be used as a prefix (--y) or a postfix (y--) operator.
- When used as a prefix operator, the variable is decremented first, and then the updated value is used.

Example: `int y = 5;`

`int r1 = --y; //r1=4 and y=4`

- When used as a postfix operator, the current value of the variable is used first, and then the variable is decremented.

Example: `int y = 10;`

`int r2 = y--; //r2=10 and y=9`

7. Write down the differences increment and decrement operators. [2023]

Answer: Differences between increment and decrement operators are given below:

Aspect	Increment Operator	Decrement Operator
Definition	The increment operator increases the value of a variable by 1.	The decrement operator decreases the value of a variable by 1.
Symbol	Represented by double plus signs (++).	Represented by double minus signs (--).
Operation	Adds 1 to the operand.	Subtracts 1 from the operand.
Usage	Can be used as prefix (++x) or postfix (x++).	Can be used as prefix (--x) or postfix (x--).
Example	Prefix Increment: <code>int x = 5;</code> <code>int M = ++x; //M = 6 and x = 6</code> Postfix Increment: <code>int y = 5;</code> <code>int N = y++; //N = 5 and y = 6</code>	Prefix Decrement: <code>int x = 5;</code> <code>int M = --x; //M = 4 and x = 4</code> Postfix Decrement: <code>int y = 5;</code> <code>int N = y--; //N = 5 and y = 4</code>

8. Distinguish between `x++` and `++x`. [2022, 2017]

Answer: Differences between `x++` and `++x` are given below:

Aspect	<code>x++</code>	<code>++x</code>
Increment type	Postfix increment.	Prefix increment.
Effect on x	Evaluates 'x' and then increments its value.	Increments 'x' first and then evaluates its new value.
Value used in expression	The original value of 'x' is used in the expression.	The updated value of 'x' is used in the expression.
Return value	The value of 'x' before increment is returned.	The value of 'x' after increment is returned.
Example	<pre>int x = 5; int r1 = x++;</pre> After execution, the values of 'x' and 'r1' will be 6 and 5 respectively.	<pre>int x = 5; int r2 = ++x;</pre> After execution, both of the values of 'x' and 'r2' will be 6.

9. Distinguish between `--a` and `a--`. [2018]

Answer: Differences between `--a` and `a--` are given below:

Aspect	<code>--a</code>	<code>a--</code>
Decrement type	Prefix decrement.	Postfix decrement.
Effect on a	Decrements 'a' first and then evaluates its new value.	Evaluates 'a' and then decrements its value.
Value used in expression	The updated value of 'a' is used in the expression.	The original value of 'a' is used in the expression.
Return value	The value of 'a' after decrement is returned.	The value of 'a' before decrement is returned.
Example	<pre>int a = 5; int r1 = --a;</pre> After execution, both of the values of 'a' and 'r1' will be 4.	<pre>int a = 5; int r2 = a--;</pre> After execution, the values of 'a' and 'r2' will be 4 and 5 respectively.

NB: This notebook is created using several reference books of C, ChatGPT, and other external sources. Please let me know if you find any mistakes. Thank you.